PYTHON IN DATABRICKS CHEAT SHEET

Production Reference: DBUtils, Delta Live Tables (DLT), pandas on Spark, SDK & Magic Commands

PYTHON 3.10+ | DBUTILS | DELTA LIVE TABLES | DATABRICKS SDK

dbutils Master Reference (fs, secrets, notebook)

Databricks Utilities

```python
# 1. File System Operations (Cloud Storage Navigation)
files = dbutils.fs.ls("/Volumes/prod_catalog/raw_landing/")
dbutils.fs.mv("/tmp/source.csv", "/mnt/datalake/destination.csv")

# 2. Secret Management (Azure Key Vault / AWS Secrets Manager)
api_token = dbutils.secrets.get(scope="kv_fin_prod", key="payment_gateway_key")

# 3. Parameterized Widgets (Pass arguments to notebooks)
dbutils.widgets.text("env", "prod", "Environment")
current_env = dbutils.widgets.get("env")

# 4. Notebook Workflow Orchestration & Parameter Passing
result = dbutils.notebook.run("sub_etl", timeout_seconds=1800, arguments={"batch_id":
"102"})
dbutils.notebook.exit('{"status": "SUCCESS", "records": 45000}')
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **dbutils.fs:** Interacts directly with ADLS/S3 storage drivers, abstracting distributed cloud credentials and API calls.
- **dbutils.secrets.get():** Redacts sensitive API keys and database credentials automatically from notebook outputs and cluster logs.
- **dbutils.widgets:** Injects runtime parameters into notebook cells from Airflow, Azure Data Factory, or Databricks Workflows.
- **dbutils.notebook.exit():** Serializes structured JSON return payloads back to parent workflow orchestrators.

Security Hardening: Never hardcode secrets in notebooks; always configure Unity Catalog Secret Scopes with Key Vault backing.

Delta Live Tables (DLT) Declarative Pipelines

Declarative ETL

```python
import dlt
from pyspark.sql.functions import col, expr

@dlt.table(
    name="bronze_raw_txns",
    comment="Raw incremental streaming transactions"
)
def bronze_txns():
    return (spark.readStream.format("cloudFiles")
            .option("cloudFiles.format", "json")
            .load("/Volumes/prod/raw/txns"))

@dlt.table(name="silver_clean_txns")
@dlt.expect_or_drop("valid_amount", "amount > 0")
@dlt.expect_or_fail("valid_id", "txn_id IS NOT NULL")
def silver_txns():
    return dlt.read_stream("bronze_raw_txns").filter(col("status") == "SETTLED")
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **@dlt.table:** Declares a managed Delta table; DLT constructs the global DAG dependency graph and manages cluster provisioning.
- **dlt.read_stream():** Binds streaming table dependencies, propagating schema changes and updates through the Medallion architecture.
- **@dlt.expect_or_drop:** Enforces data quality rules; drops invalid rows while recording violation metrics in the pipeline event log.
- **@dlt.expect_or_fail:** Halts pipeline execution immediately if critical schema or business invariants are breached.

Serverless DLT: Serverless DLT pipelines eliminate cluster spin-up time and dynamically allocate compute as streaming queues fluctuate.

Streaming CDC with dlt.apply_changes() (SCD 1 & 2)

Change Data Capture

```python
import dlt

# Declare destination streaming table
dlt.create_streaming_table("silver_dim_customers")

# Turnkey Slowly Changing Dimension (SCD Type 2)
dlt.apply_changes(
    target="silver_dim_customers",
    source="bronze_cdc_customers",
    keys=["customer_id"],
    sequence_by="operation_timestamp",
    apply_as_deletes=col("op_type") == "D",
    except_column_list=["op_type", "ingested_at"],
    stored_as_scd_type=2 # 1 = In-Place Overwrite; 2 = Historical Tracking
)
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **dlt.apply_changes():** Native declarative CDC engine supporting out-of-order Debezium / Fivetran / Oracle GoldenGate events.
- **sequence_by:** Monotonic sequence key or timestamp ensuring out-of-order CDC packets are resolved deterministically.
- **stored_as_scd_type=2:** Automatically maintains validity tracking columns: `__START_AT`, `__END_AT`, and active flags.
- **apply_as_deletes:** Evaluates soft or hard delete operations, setting `__END_AT` without physically corrupting history.

SCD Type 1 vs 2: Use SCD 1 for current-state lookup dimensions; use SCD 2 for point-in-time compliance and financial audits.

Pandas API on Spark (pyspark.pandas as ps)

Distributed Pandas

```python
import pyspark.pandas as ps

# 100% Pandas Syntax executed across distributed Spark Cluster!
psdf = ps.read_parquet("abfss://silver@adls.dfs.core.windows.net/orders")

# High-level analytical calculations with no PySpark boilerplate
psdf["tax"] = psdf["amount"] * 0.18
summary = psdf.groupby("region")["amount"].agg(["mean", "sum", "std"])

# Seamless Two-Way Conversion
spark_df = psdf.to_spark()
ps_df_again = spark_df.to_pandas_on_spark()
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Distributed Pandas:** Translates standard Pandas method invocations directly into distributed Spark Catalyst query execution plans.
- **No Driver Memory Limit:** Unlike native Pandas, data remains distributed across worker executor memory; processes 100GB+ seamlessly.
- **to_spark() / to_pandas_on_spark():** Zero-copy conversion between PySpark DataFrames and Pandas-on-Spark representations.
- **Index Management:** Uses default distributed indexing; avoid setting sequential range indexes on huge datasets to prevent sorting shuffles.

Anti-Pattern: Avoid calling `.to_pandas()` or `.iloc` on large DataFrames as it pulls all distributed data into driver RAM, risking OOM!

Databricks Connect v2 & Databricks Python SDK

Developer Tooling

```python
# Databricks Connect v2 (Local IDE to Remote Cluster over gRPC)
from databricks.connect import DatabricksSession
spark = DatabricksSession.builder.getOrCreate()
df = spark.read.table("prod_catalog.silver.transactions")
df.filter("amount > 1000").show()

# Databricks Python SDK (Workspace Automation)
from databricks.sdk import WorkspaceClient
w = WorkspaceClient()
run = w.jobs.run_now(job_id=892145)
clusters = w.clusters.list()
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **Databricks Connect v2:** Built on the open-source Spark Connect protocol; local Python code runs thin clients over secure gRPC to remote clusters.
- **Local IDE Debugging:** Set breakpoints in VS Code or PyCharm, step through code interactively, and execute against production cluster data.
- **WorkspaceClient:** Official SDK managing all Databricks REST API endpoints: Jobs, Clusters, Unity Catalog, Warehouses, and Secrets.
- **Unified Authentication:** Automatically resolves credentials from environment variables (`DATABRICKS_HOST`, `DATABRICKS_TOKEN`) or Azure CLI.

Thin Client: Databricks Connect v2 sends unresolved DataFrame plans via gRPC; execution runs entirely on the remote Databricks cluster.

Databricks Magic Commands & Scoping

Notebook Magic

```python
# 1. SQL Magic & Accessing Results in Python
%sql
SELECT customer_id, SUM(amount) AS total FROM silver.orders GROUP BY customer_id

# Access SQL query output in next Python cell:
sql_df = _sqldf.filter("total > 5000")

# 2. Ephemeral Environment Setup
%pip install pydantic==2.8.2 tenacity==8.4.1

# 3. Modular File Execution
%run ./common_utilities_notebook
```

CODE BREAKDOWN & ENGINE MECHANICS:

- **%sql & _sqldf:** Running a `%sql` cell automatically binds the output DataFrame to the `_sqldf` variable in downstream Python cells.
- **%pip:** Installs isolated dependencies onto the cluster driver and worker environments for the duration of the notebook session only.
- **%run:** Executes external shared notebooks in-line, inheriting functions, imports, and variables directly into the current notebook session.
- **%sh & %fs:** Executes bash commands inside the driver container and navigates storage volumes via command-line utilities.

%run vs dbutils.notebook.run: `%run` shares memory & variables in the same session; `dbutils.notebook.run` executes in an isolated ephemeral job context.

🐍 PYTHON IN DATABRICKS CHEAT SHEET

Wheels, Databricks Asset Bundles (DABs), Observability, Resilience, Types & Unit Testing

DABS / CI/CD | PRODUCTION WHEELS | OBSERVABILITY

📦 Wheels & Unity Catalog Volumes

Packaging

```
# In-Notebook ephemeral dependency install
%pip install pydantic==2.8.2 tenacity==8.4.1

# Production Wheel Distribution via Unity Catalog Volumes
# Store built wheel in governed volume:
# /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl
%pip install /Volumes/prod_catalog/core_libs/my_etl-1.2.0-py3-none-any.whl

# Import modular reusable package in notebook
from my_etl.transformers import BankingNormalizer
normalizer = BankingNormalizer(currency="EUR")
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Unity Catalog Volumes:** Governed storage locations in cloud storage supporting file-level RBAC, audit logging, and binary package versioning.
- **Custom Wheels (.whl):** Package enterprise business logic into modular Python wheels to enforce automated CI/CD unit testing and release tagging.
- **Cluster Libraries:** Attach volume paths to cluster configurations or Job task dependencies to pre-install packages across worker nodes.
- **Zero DBFS Root:** Completely replaces insecure legacy DBFS root paths with enterprise-governed Unity Catalog storage locations.

💡 **Volumes for Libraries:** Replacing legacy DBFS root with Unity Catalog Volumes ensures strict access control and auditing for third-party binary artifacts.

🚀 Databricks Asset Bundles (DABs) CI/CD

DevOps

```
# databricks.yml definition snippet
bundle:
  name: bfsi_settlement_pipeline

targets:
  dev:
    workspace:
      host: https://adb-dev.azuredatabricks.net
  prod:
    workspace:
      host: https://adb-prod.azuredatabricks.net

resources:
  jobs:
    nightly_settlement:
      name: "[prod] Nightly Settlement Job"
      tasks:
        - task_key: run_dlt
          pipeline_task:
            pipeline_id: 489a-bc12-dlt-id
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Infrastructure-as-Code (IaC):** Declaratively defines Databricks workflows, notebooks, DLT pipelines, and permissions in source-controlled YAML.
- **Multi-Environment:** `targets: dev / prod` abstracts workspace host URLs and cluster settings, enabling clean promotion across stages.
- **CLI Commands:** `databricks bundle validate` (linting), `bundle deploy -t prod` (deployment), and `bundle run` (execution).
- **GitHub Actions / Azure DevOps:** Integrates directly into CI/CD runners using service principals without manual workspace UI clicking.

⚡ **Modern DevOps:** Databricks Asset Bundles (DABs) are the standard for deploying enterprise data engineering projects in 2025/2026.

📊 Structured Logging & Observability

Observability

```
import json, logging
from datetime import datetime

class StructuredPipelineLogger:
    def __init__(self, pipeline_name: str):
        self.name = pipeline_name

    def log_event(self, event_name: str, records: int, duration_sec: float):
        payload = {
            "timestamp": datetime.utcnow().isoformat(),
            "pipeline": self.name,
            "event": event_name,
            "records_processed": records,
            "duration_sec": duration_sec
        }
        print(json.dumps(payload)) # Ingested into Azure Log Analytics

audit = StructuredPipelineLogger("SEPA_Payment_Processor")
audit.log_event("STAGE_COMPLETED", records=845000, duration_sec=14.2)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Single-Line JSON:** Emitting serialized JSON to standard stdout ensures cloud observability scrapers (Datadog, Splunk, Azure Log Analytics) parse attributes automatically.
- **Metrics Tracking:** Records throughput metrics, processing duration, and stage identifiers for SLI/SLA compliance monitoring.
- **Distributed Driver Scoping:** Instantiated on the driver node to capture macro-level stage completions and pipeline milestones.
- **Alerting Triggers:** Enables automated incident triggers when duration spikes above historical 95th percentile thresholds.

🔒 **Enterprise Standard:** Standardized JSON logging links cluster run IDs and pipeline commits directly into enterprise monitoring dashboards.

🔄 Concurrent Extraction & Exponential Retries

Resilience

```
from concurrent.futures import ThreadPoolExecutor
from tenacity import retry, stop_after_attempt, wait_exponential

@retry(stop=stop_after_attempt(3), wait=wait_exponential(multiplier=1, min=2, max=10))
def fetch_api_page(page_id: int) -> dict:
    # Resilient REST API caller with exponential backoff
    return {"page": page_id, "data": [1, 2, 3]}

# Parallelize lightweight I/O on Driver before Spark processing
with ThreadPoolExecutor(max_workers=8) as executor:
    results = list(executor.map(fetch_api_page, range(1, 50)))

df_api = spark.createDataFrame(results)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **Driver Multithreading:** Uses Python `ThreadPoolExecutor` on the driver node to parallelize I/O-bound REST API web requests efficiently.
- **tenacity @retry:** Catches transient HTTP 429 rate-limiting and 503 gateway timeout exceptions, backing off exponentially before retrying.
- **createDataFrame():** Converts accumulated JSON page dictionaries into a distributed PySpark DataFrame across worker executors.
- **Separation of Concerns:** Use Python threads for I/O fetching; use Spark distributed partitions for data transformations and writes.

⚡ **Thread Concurrency:** Use Python `ThreadPoolExecutor` on the driver for I/O-bound REST calls, then parallelize the dataset with `spark.createDataFrame()`.

🛡️ Type Annotations & Schema Contracts

Data Quality

```
from pyspark.sql import DataFrame, Column
from typing import Callable

def apply_curation(
    df: DataFrame,
    primary_key: str,
    business_rule: Callable[[Column], Column]
) -> DataFrame:
    # Pre-flight contract validation
    assert primary_key in df.columns, f"Missing required key: {primary_key}"
    return df.filter(business_rule(df[primary_key]))
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **PEP 484 Typing:** Explicitly annotates PySpark DataFrames, Columns, and function callables to enable static type analysis with `mypy`.
- **Pre-Flight Assertions:** Validates input schemas before submitting heavy distributed compute jobs, failing in 10ms instead of 2 hours later.
- **Higher-Order Parameterization:** Accepts Column lambda functions (`Callable[[Column], Column]`) for modular rule composition.
- **Documentation Integrity:** Serves as self-documenting code contracts across enterprise data engineering squads.

💡 **Fail Fast:** Pre-flight contract validation prevents pipelines from burning expensive compute hours on downstream schema errors.

🧪 Unit Testing PySpark Pipelines in CI/CD

Testing

```
import pytest
from pyspark.testing.utils import assertDataFrameEqual

def test_filter_active(spark):
    input_data = [(1, "ACTIVE"), (2, "CANCELLED")]
    input_df = spark.createDataFrame(input_data, ["id", "status"])

    expected_data = [(1, "ACTIVE")]
    expected_df = spark.createDataFrame(expected_data, ["id", "status"])

    result_df = input_df.filter("status = 'ACTIVE'")
    # Spark 3.5+ built-in test assertion
    assertDataFrameEqual(result_df, expected_df)
```

🔍 CODE BREAKDOWN & ENGINE MECHANICS:

- **assertDataFrameEqual:** Built into PySpark 3.5+; compares distributed DataFrames, ignoring row ordering and providing detailed schema diffs.
- **Tolerance Support:** Supports `atol` and `rtol` arguments to accommodate floating-point precision differences across environments.
- **Zero External Dependencies:** Eliminates third-party test libraries (e.g. `chispa`), simplifying GitHub Actions / Azure DevOps CI runners.
- **Pytest Fixtures:** Pass a shared session-scoped `spark` fixture to run unit test suites locally in under 5 seconds.

⚡ **CI/CD Speed:** Run PySpark unit tests using local mode (`local[2]`) in CI runners to validate logic prior to deploying Asset Bundles.